ChatGPT

```
import java.util.ArrayList;

class Solution {

    private void mergeSort1(int[] nums, int low, int high) {
        if (low >= high) {
            return;
        }

        int mid = (low + high) / 2;

        mergeSort1(nums, low, mid);
        mergeSort1(nums, mid + 1, high);

        merge(nums, low, mid, high);
    }

    private void merge(int[] nums, int low, int mid, int high) {
        ArrayList<Integer> temp = new ArrayList<>();

        int left = low;
        int right = mid + 1;

        while (left <= mid && right <= high) {
            if (nums[left] <= nums[right]) {
                temp.add(nums[left]);
                left++;
            } else {
                temp.add(nums[right]);
                right++;
            }
        }

        while (left <= mid) {
            temp.add(nums[left]);
            left++;
        }

        while (right <= high) {
            temp.add(nums[right]);
            right++;
        }

        for (int i = low; i <= high; i++) {
            nums[i] = temp.get(i - low);
```

```
        }
    }

    public int[] mergeSort(int[] nums) {
        int n = nums.length;

        int low = 0;
        int high = n - 1;

        mergeSort1(nums, low, high);

        return nums;
    }
}
```

Teach me Merge Sort
Show moreShow less

Compare with other AI

Absolutely. Your code is a **top-down Merge Sort** implementation. Let's understand the idea first, then walk through your exact code.

1. The big idea

Merge Sort follows one simple strategy:

> **Divide the array into smaller pieces, sort those pieces, then merge them back together.**

For example:

```
[5, 2, 4, 1, 3]

        [5, 2, 4, 1, 3]
              /      \
       [5, 2, 4]    [1, 3]
         /    \       / \
     [5,2]   [4]   [1] [3]
      / \
    [5] [2]
```

Eventually every piece contains just **one element**.

A single element is already sorted.

Then we merge:

```
[5] + [2] → [2, 5]

[2, 5] + [4] → [2, 4, 5]

[1] + [3] → [1, 3]

[2, 4, 5] + [1, 3]
→ [1, 2, 3, 4, 5]
```

That's Merge Sort.

2. There are really TWO operations

It's useful to separate Merge Sort into:

A. Divide

Keep splitting:

```
[5, 2, 4, 1, 3]
        ↓
[5, 2, 4]    [1, 3]
        ↓
[5, 2] [4]    [1] [3]
        ↓
[5] [2] [4] [1] [3]
```

B. Merge

Then combine sorted pieces:

```
[5] + [2]
     ↓
  [2, 5]
[2, 5] + [4]
        ↓
  [2, 4, 5]
[1] + [3]
     ↓
  [1, 3]
[2, 4, 5] + [1, 3]
           ↓
  [1, 2, 3, 4, 5]
```

Your code has exactly these two parts:

```
Java
mergeSort1(...)
```

does the **dividing**, while

```
Java
merge(...)
```

does the **merging**.

3. Understanding `mergeSort1`

Your function:

```java
private void mergeSort1(int[] nums, int low, int high) {
    if (low >= high) {
        return;
    }

    int mid = (low + high) / 2;

    mergeSort1(nums, low, mid);
    mergeSort1(nums, mid + 1, high);

    merge(nums, low, mid, high);
}
```

Let's understand each line.

Base case

```java
if (low >= high) {
    return;
}
```

This is extremely important.

Suppose:

```
low = 3
high = 3
```

That means we're looking at:

```
nums[3...3]
```

There is only **one element**.

There's nothing left to sort.

So we stop.

In other words:

> If the section contains 0 or 1 elements, it's already sorted.

---

4. Finding the middle

```java
int mid = (low + high) / 2;
```

Suppose:

```
low = 0
high = 4
```

Then:

```
mid = (0 + 4) / 2
    = 2
```

So we divide:

```
0   1   2 | 3   4
[5, 2, 4 | 1, 3]
          ↑
         mid
```

The two halves are:

```
low ... mid
```

and

```
mid + 1 ... high
```

So:

```
[0 ... 2]    [3 ... 4]
```

5. The two recursive calls

These two lines are the heart of the **divide** phase:

```java
mergeSort1(nums, low, mid);
mergeSort1(nums, mid + 1, high);
```

The first says:

> Sort the left half.

The second says:

> Sort the right half.

For:

```
[5, 2, 4, 1, 3]
```

we get:

```
mergeSort1(0, 2)
mergeSort1(3, 4)
```

And those calls keep splitting.

Eventually:

```
[5] [2] [4] [1] [3]
```

Now all recursive calls hit:

```java
if (low >= high)
    return;
```

and start returning.

---

6. The surprising part: merging happens AFTER recursion

Look at this:

```java
mergeSort1(nums, low, mid);
mergeSort1(nums, mid + 1, high);

merge(nums, low, mid, high);
```

Notice that `merge()` happens **after** both recursive calls.

This is what makes Merge Sort work.

Imagine:

```
[5, 2, 4, 1, 3]
```

The algorithm doesn't immediately try to sort the entire array.

Instead:

```
                [5,2,4,1,3]
               /           \
         [5,2,4]           [1,3]
         /    \             /  \
      [5,2]   [4]         [1] [3]
      /  \
    [5]  [2]
```

Then recursion returns upward.

That's when merging starts.

---

7. Now let's understand `merge()`

Your function:

```java
private void merge(int[] nums, int low, int mid, int high) {
    ArrayList<Integer> temp = new ArrayList<>();

    int left = low;
```

```
    int right = mid + 1;
```

The important idea is that **`merge()`** assumes:

```
left half is already sorted
right half is already sorted
```

For example:

```
[2, 5] | [1, 4]
  ↑         ↑
 left      right
```

Our job is to combine them into:

```
[1, 2, 4, 5]
```

8. Why do we need `left` and `right`?

You have:

```java
int left = low;
int right = mid + 1;
```

Suppose:

```
[2, 5] [1, 4]
```

Then:

```
left  → 2
right → 1
```

We compare:

```
2 vs 1
```

The smaller value is **`1`**.

So:

```
temp = [1]
```

and move **`right`**:

```
left  → 2
right → 4
```

Compare:

```
2 vs 4
```

Take **2**:

```
temp = [1, 2]
```

Move **left**:

```
left  → 5
right → 4
```

Compare:

```
5 vs 4
```

Take **4**:

```
temp = [1, 2, 4]
```

Then:

```
right
```

has nothing left.

So we copy the remaining **5**.

Final:

```
[1, 2, 4, 5]
```

9. Understanding your main while loop

This is the most important part:

```java
while (left <= mid && right <= high) {
    if (nums[left] <= nums[right]) {
        temp.add(nums[left]);
        left++;
    } else {
        temp.add(nums[right]);
        right++;
    }
}
```

Read it almost like English:

> As long as both halves still have elements, compare their current elements and take the smaller one.

For example:

```
Left half:   [2, 5]
             ↑

Right half:  [1, 4]
             ↑
```

Compare:

```
2 <= 1 ? No
```

So:

```
Java
temp.add(nums[right]);
right++;
```

Now:

```
temp = [1]

Left:   [2, 5]
          ↑

Right:  [1, 4]
             ↑
```

Compare:

```
2 <= 4 ? Yes
```

So:

```
temp = [1, 2]
```

and move **`left`**.

Eventually:

```
temp = [1, 2, 4, 5]
```

10. Why are there TWO more while loops?

After the main loop:

```
Java
while (left <= mid && right <= high)
```

one side might still have elements.

For example:

```
Left:  [2, 3, 8]
             ↑
Right: [4]
          ↑
```

We take **2**, **3**, **4**.

Now:

```
Left:  [2, 3, 8]
                ↑
```

```
Right: [4]
          DONE
```

The main loop stops because the right side is finished.

But `8` still needs to be copied.

That's why you have:

```java
while (left <= mid) {
    temp.add(nums[left]);
    left++;
}
```

Similarly, the right side might have leftovers:

```java
while (right <= high) {
    temp.add(nums[right]);
    right++;
}
```

11. Why copy `temp` back into `nums`?

At this point:

```
temp = [1, 2, 4, 5]
```

But `nums` hasn't necessarily been updated yet.

So:

```java
for (int i = low; i <= high; i++) {
    nums[i] = temp.get(i - low);
}
```

This puts the sorted values back into the original array.

The slightly confusing part is:

```java
temp.get(i - low)
```

Why `i - low`?

Suppose:

```
low = 2
high = 5
```

and:

```
temp = [1, 2, 4, 5]
```

We want:

```
nums[2] = temp[0]
nums[3] = temp[1]
nums[4] = temp[2]
nums[5] = temp[3]
```

Therefore:

```
temp index = nums index - low
```

Hence:

```
Java
temp.get(i - low)
```

12. Let's trace your entire algorithm

Take:

```
[5, 2, 4, 1, 3]
```

Initially:

```
Java
mergeSort(nums, 0, 4)
```

Divide

```
[5, 2, 4, 1, 3]
          |
          ↓
[5, 2, 4]     [1, 3]
```

Then:

```
[5, 2] [4]     [1] [3]
```

Then:

```
[5] [2] [4] [1] [3]
```

Merge upward

First:

```
[5] + [2]
↓
[2, 5]
```

Then:

```
[2, 5] + [4]
```

```
↓
[2, 4, 5]
```

And:

```
[1] + [3]
↓
[1, 3]
```

Finally:

```
[2, 4, 5] + [1, 3]
```

Compare:

```
2 vs 1 → take 1
2 vs 3 → take 2
4 vs 3 → take 3
4 vs nothing → take 4
5 vs nothing → take 5
```

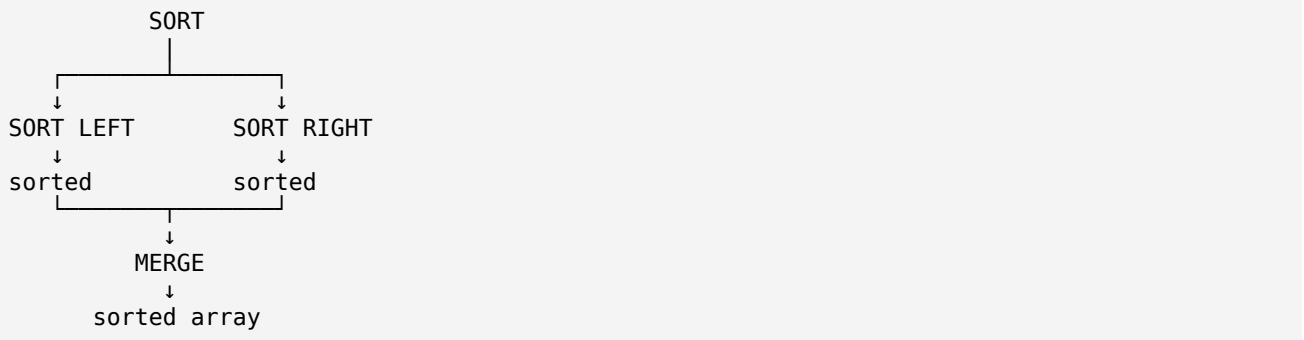
Result:

```
[1, 2, 3, 4, 5]
```

13. The mental model I want you to remember

Don't try to memorize the code line-by-line.

Remember this:



Or even simpler:

> **Split → Split → Split → Merge → Merge → Merge**

---

14. Why is Merge Sort O(n log n)?

There are two things happening.

How many levels of splitting?

Every time we split in half:

```
n
n/2
n/4
n/8
...
1
```

This gives approximately:

```
log₂(n)
```

levels.

How much work per level?

At every level, the elements eventually get processed during merging.

That's approximately:

```
n
```

work per level.

Therefore:

```
n × log n
```

So:

Time complexity:

```
O(n log n)
```

And because your implementation creates a temporary list during merging:

Space complexity:

```
O(n)
```

15. One small improvement to your code

This:

```Java
int mid = (low + high) / 2;
```

is normally fine for coding interviews, but the safer version is:

```Java
int mid = low + (high - low) / 2;
```

It avoids integer overflow when **`low + high`** becomes too large.

Also, for Merge Sort, an `ArrayList<Integer>` works, but an `int[]` temporary array is generally more efficient because `ArrayList<Integer>` involves boxing `int` values into `Integer` objects.

The entire algorithm in one sentence

Merge Sort recursively divides an array into single-element pieces, then repeatedly merges two sorted pieces by comparing their smallest remaining elements.

If you're learning this for DSA/interviews, the **next thing to master is the `merge()` function**—once you can manually merge `[2,5,8]` and `[1,3,7]` without looking at the code, Merge Sort becomes much easier to understand.